Ubuntu・OS基礎 練習課題と解答例

P0〜P6 課題と操作手順

Ubuntu Server 24.04 LTS
JDU Cyber Security / 2026年度教材

Ubuntu and OS Foundations — 練習課題と解答例

注: ロシア語版・ウズベク語版は現行のP0〜P6と内容を同期した翻訳初稿です。母語話者による校閲は未実施です。採点条件の正本は日本語版です。

対象: 練習課題P0〜P6

環境: Ubuntu Server 24.04 LTS

進め方: まず各Pの「課題」を読む。自分で試した後、「解答例」の手順で操作と考え方を確認する。同じ技能を使うMission Mは自力で解く。

最初にUbuntuへ接続する

AWS Academy Learner Labのセットアップを完了してから、AWSコンソールでCloudShellを開く。CloudShellで次を実行して、自分のUbuntu演習サーバーへ接続する。

```
ssh jdu-ubuntu
```

P0〜P5は、接続先のUbuntuで進める。CloudShell側の操作が必要なP6では、手順中に実行場所を示す。

0. 共通ルール

P0〜P6は練習用である。P0はOSの観察を手順付きで行う導入練習である。M1〜M6とは別のdirectory、user、group、service、portを使う。M7は統合課題であるため、対応する練習課題を設けない。

各Pには、最初に問題文、その後に解答例を置く。問題文だけを読んで解いてもよい。解答例の手順を見ながら進めてもよい。PとMの採点対象は別であり、PのPASSはMのPASSにならない。

Ubuntuで次を実行する。表示が`ssm-user`であることを確認する。

```
id -un
```

別のuser名が表示された場合は、`exit`を1回実行する。もう一度確認する。

```
exit
id -un
```

各手順では、最初に作業directoryへ移動する。移動後に`pwd`で現在地を確認する。

Ubuntu側の判定は、`ssm-user`で次の形で実行する。P6のCloudShell側の判定は、その環境で実行する。

```
jdu-check P1
```

判定は課題の完成状態を調べる。観察しただけの操作や途中の操作は、判定項目に含まれない場合がある。判定結果は設定済みの教員用進捗画面へ自動送信される。送信できなくても、UbuntuまたはCloudShell上のPASS/FAILは確認できる。

最初からやり直す場合だけresetする。

```
jdu-reset P1
```

P0 Environment and OS

学ぶこと

- UbuntuというOSとLinux kernelの情報を区別する。
- 現在のユーザー、ホスト名、PID 1のプロセスを実機で確認する。
- コマンドの結果をファイルへ記録し、保存・確認・提出する。

課題

自分のUbuntu演習サーバーから6項目を調べ、`~/jdu-lab/p0/observation.env`へ記録する。以前のM0を、手順付き練習P0に変更したものである。値は以下の例や他の学生からコピーせず、自分のサーバーの表示を使う。6項目を`jdu-check P0`で確認する。

解答例（操作手順）

手順1: 接続先とユーザーを確認する

この資料の冒頭のとおり、CloudShellで`ssh jdu-ubuntu`を実行する。以下の手順はすべて接続先のUbuntuで行う。

```
# ホームディレクトリへ移動する。
cd ~
# 現在のディレクトリを表示する。
pwd
# 現在のユーザー名を表示する。
id -un
```

`id -un`の表示が`ssm-user`であることを確認する。これが後で記録する`USER_NAME`である。以降の表示はメモしておく。

手順2: OSの種類とバージョンを確認する

```
# OSの種類とバージョンを表示する。
cat /etc/os-release
```

`ID=`の値を`OS_ID`、`VERSION_ID=`の値を`OS_VERSION_ID`としてメモする。`NAME`や`PRETTY_NAME`とは異なる項目である。表示に引用符がある場合は、値だけを記録し、引用符は付けない。

手順3: Linux kernelのバージョンを確認する

```
# Linuxカーネルのリリース番号を表示する。
uname -r
```

`-r`はkernelのreleaseを表示する指定である。表示された文字列全体を`KERNEL_RELEASE`としてメモする。OSのバージョンとkernelのバージョンは別の情報である。

手順4: PID 1のプロセス名を確認する

```
# PID 1のプロセス名を表示する。
cat /proc/1/comm
```

`/proc`には実行中のシステムの情報が公開されている。`1`はプロセス番号（PID）、`comm`はそのプロセスの名前を確認するファイルである。表示を`PID1_COMM`としてメモする。

手順5: ユーザー名とホスト名を確認する

```
# 現在のユーザー名を表示する。
id -un
# サーバーのホスト名を表示する。
hostname
```

1つ目をUSER_NAME、2つ目をHOST_NAMEとしてメモする。ホスト名はサーバーを識別する名前であり、ユーザー名や提出用Server IDとは別である。

手順6: 記録用ファイルを作成する

```
# ホームディレクトリへ移動する。
cd ~
# P0の作業ディレクトリを作る。
mkdir -p jdu-lab/p0
# P0の作業ディレクトリへ移動する。
cd jdu-lab/p0
# 現在地を表示する。
pwd
# 記録ファイルを編集する。
nano observation.env
```

nanoが開いたら、次の6行を入力し、それぞれの=の右にメモした値を入れる。この枠は編集するファイルの内容であり、コマンドとして実行しない。

```
OS_ID=
OS_VERSION_ID=
KERNEL_RELEASE=
PID1_COMM=
USER_NAME=
HOST_NAME=
```

各項目を1行にし、項目名や=の前後に空白を入れない。値に引用符は付けない。Ctrl+Oで保存し、ファイル名がobservation.envであることを確認してEnterを押す。Ctrl+Xでエディタを閉じる。

手順7: 保存した内容を確認する

```
# 現在地を表示する。
pwd
# 記録ファイルの存在と権限を確認する。
ls -l observation.env
# 記録した6項目を表示する。
cat observation.env
```

6行すべてに値があり、手順1〜5の出力と一致するか確認する。空欄や誤記があればnano observation.envで修正し、再び保存する。

手順8: 判定・提出する

```
# 採点に使うユーザー名を確認する。
id -un
# P0の完成状態を採点する。
jdu-check P0
```

6件すべてがPASS、結果が6 / 6 checks clearedになればP0の記録は完成である。FAILなら指摘された項目の実機情報を再確認し、ファイルを修正して再実行する。何度でも実行できる。

RESULTは課題の判定、REPORTは教員サーバーへの送信結果である。送信に失敗した場合はファイルを消したりresetしたりせず、通信・登録の状態を確認して再実行する。

手順9: 次の練習へ進む

```
# 次の練習に備えてホームディレクトリへ戻る。
cd ~
```

P1へ進む。CloudShellへ戻りたい場合だけexitを実行する。初回のP0ではreset不要である。P0の記録を消してやり直す場合だけ、Ubuntuでjdu-reset P0を実行する。

P1 Shell、path、file、text

学ぶこと

- Home directoryと作業directoryを区別する。
- Directoryを作る。
- Fileをコピーする。
- Logから必要な行を保存する。

課題

~/jdu-lab/p1/inboxに設定ファイルとログがある。~/jdu-lab/p1/practice01は未完成で、不要な.tmpファイルが残っている。次の状態に仕上げる。

1. practice01の下にconfig、logs、notesの3つのディレクトリを作る。
2. inbox/config/training.confをpractice01/config/training.confへ、inbox/logs/practice.logをpractice01/logs/practice.logへコピーする。元ファイルとコピーの内容を変えない。
3. practice01以下の.tmpファイルをすべて削除する。それ以外のファイルは消さない。
4. 完成したpractice01以下のディレクトリとファイルの所有者を、Ubuntuの管理ユーザーssm-userにする。
5. コピーしたログからWARNを含む行だけを、行番号を付けずにpractice01/notes/warnings.txtへ保存する。
6. 同じログの最後の4行を、元の順序でpractice01/notes/recent.txtへ保存する。

完成状態をjdu-check P1で判定する。6項目すべてがPASSなら終了する。

解答例（操作手順）

手順1: Home directoryへ移動する

```
# ホームディレクトリへ移動する。
cd ~
# 現在地を表示する。
pwd
```

表示が/home/ssm-userであることを確認する。

手順2: P1 directoryへ移動する

```
# P1の作業ディレクトリへ移動する。
cd ~/jdu-lab/p1
# 現在地を表示する。
pwd
# 元ファイルと未完成の提出先を木構造で表示する。
tree
```

`inbox`と、未完成の`practice01`を確認する。

手順3: 必要なdirectoryを作る

```
# P1の作業ディレクトリへ移動する。
cd ~/jdu-lab/p1
# 設定ファイル用ディレクトリを作る。
mkdir -p practice01/config
# ログ用ディレクトリを作る。
mkdir -p practice01/logs
# 結果ファイル用ディレクトリを作る。
mkdir -p practice01/notes
# 作成した構成を表示する。
tree practice01
```

手順4: Config fileをコピーする

コピー先のdirectoryへ移動する。

```
# コピー先の設定ディレクトリへ移動する。
cd ~/jdu-lab/p1/practice01/config
# 現在地を確認する。
pwd
# 元の設定ファイルを現在のディレクトリへコピーする。
cp ../../inbox/config/training.conf .
# コピーしたファイルを表示する。
ls -l
```

`.`は現在のdirectoryを表す。

手順5: Log fileをコピーする

```
# コピー先のログディレクトリへ移動する。
cd ~/jdu-lab/p1/practice01/logs
# 現在地を確認する。
pwd
# 元のログを現在のディレクトリへコピーする。
cp ../../inbox/logs/practice.log .
# コピーしたファイルを表示する。
ls -l
```

手順6: 一時fileを確認して削除する

`practice01`へ移動し、削除前の構成を確認する。

```
# 完成させるディレクトリへ移動する。
cd ~/jdu-lab/p1/practice01
# 現在地を確認する。
pwd
# 削除前のファイル構成を表示する。
tree
```

treeの表示から、stagingの中にあるtraining.conf.tmpとpractice.log.tmpを探す。次に、同じ2つのfileをfindで検索する。

```
# 現在地以下の.tmpファイルだけを検索する。
find . -type f -name '*.tmp' -print
```

.は現在のdirectoryから探す指定、-type fは通常のfileだけを対象にする条件、-name '*.tmp'は名前が.tmpで終わる条件、-printは一致したパスを表示する指示である。'*.tmp'を引用符で囲む理由を含め、詳しい読み方は巻末の参照資料で説明する。

./staging/training.conf.tmpと./staging/practice.log.tmpが表示されることを確認する。表示順は問わない。検索結果とtreeの構成を照合したら、この2つの不要なfileをrmで削除する。

```
# 確認済みの不要な一時ファイル2件を削除する。
rm staging/training.conf.tmp staging/practice.log.tmp
# 削除後の構成を確認する。
tree
```

削除後のtreeで、2つの.tmpファイルが表示されないことを確認する。コピーしたconfig/training.confとlogs/practice.logが残っていることも確認する。

手順7: WARN行を確認して保存する

出力先のnotesへ移動する。まず、grepの結果を画面で確認する。

```
# 結果ファイル用ディレクトリへ移動する。
cd ~/jdu-lab/p1/practice01/notes
# 現在地を確認する。
pwd
# ログのWARNを含む行を画面に表示する。
grep 'WARN' ../logs/practice.log
```

WARNを含む行だけが表示されることを確認する。次に、同じcommandに>を追加して、結果をwarnings.txtへ保存する。>は出力先を画面からfileへ変更し、同名のfileがあれば内容を上書きする。

```
# WARN行をwarnings.txtへ保存する。
grep 'WARN' ../logs/practice.log > warnings.txt
# 保存結果を表示する。
cat warnings.txt
```

catで、先ほど画面に表示された行が保存されていることを確認する。

手順8: 最後の4行を確認して保存する

まず、tailの結果を画面で確認する。

```
# 結果ファイル用ディレクトリへ移動する。
cd ~/jdu-lab/p1/practice01/notes
# ログの最後の4行を画面に表示する。
tail -n 4 ../logs/practice.log
```

ログの最後の4行が、元の順序で表示されることを確認する。次に、>を追加して結果をrecent.txtへ保存する。

```
# 最後の4行をrecent.txtへ保存する。
tail -n 4 ../logs/practice.log > recent.txt
# 保存結果を表示する。
cat recent.txt
```

catで、先ほど画面に表示された4行が保存されていることを確認する。

手順9: 全体を確認する

```
# P1の作業ディレクトリへ戻る。
cd ~/jdu-lab/p1
# 完成したディレクトリ構成を表示する。
tree practice01
# P1の完成状態を採点する。
jdu-check P1
```

全6件がPASSになったら、M1へ進む。

P2 User、group、permission、setgid

学ぶこと

- Primary groupとsupplementary groupを確認する。
- 共有directoryへgroup permissionを設定する。
- setgidで新しいfileのgroupを継承する。

課題

共有ディレクトリ/srv/jdu-practice-shareを、書き込み担当者と閲覧担当者で使い分ける。初期状態では、jdupracticeviewerが補助グループpracticeopsに入っており、書き込み担当のjdupracticewriterは入っていない。ディレクトリとGUIDE.txtの所有者・グループ・権限も未完成である。

1. jdupracticewriterを補助グループpracticeopsに追加し、jdupracticeviewerをそのグループから削除する。
2. /srv/jdu-practice-shareの所有者とグループをroot:practiceops、権限を2775にする。新しいファイルが共有グループを引き継ぐようにする。
3. GUIDE.txtの所有者とグループをroot:practiceops、権限を664にする。
4. ssm-userからjdupracticewriterへ切り替えて、共有ディレクトリにwriter-created.txtを作る。新しいファイルのグループがpracticeopsであることを確認する。
5. ssm-userに戻り、jdupracticeviewerへ切り替える。GUIDE.txtは読めるが、共有ディレクトリに新しいファイルは作れないことを確認する。

異なるユーザーでの操作を終えたらssm-userへ戻り、jdu-check P2を実行する。グループ所属、ディレクトリ、ファイル、作成時のグループ継承、閲覧担当者のアクセスを計6項目で判定する。

採点は完成状態を確認する。手順4で作るwriter-created.txtは学習用の観察結果であり、そのファイル名を提出物として照合しない。採点時には、jdu-checkが書き込み担当者として一時ファイルを作り、そのグループがpracticeopsになるか確かめる。閲覧担当者の判定は、この作成テストが成功した後に行う。このため、作成テストが失敗すると、閲覧担当者の権限が正しくても最後の2項目がともにFAILになる。

チェックID	確認する完成状態
P2-ID-01	jdupracticewriterはpracticeopsに所属し、jdupracticeviewerは所属しない。

チェックID	確認する完成状態
P2-PERM-01	共有ディレクトリの所有者とグループはroot:practiceops。
P2-PERM-02	共有ディレクトリの権限はsetgidを含む2775。
P2-PERM-03	GUIDE.txtの所有者・グループ・権限はroot:practiceops、664。
P2-PERM-04	書き込み担当者が新規ファイルを作れ、そのグループはpracticeops。
P2-PERM-05	上の作成テストが成功し、閲覧担当者はGUIDE.txtを読めるが、新規ファイルを作れない。

最初の項目がFAILなら、手順3の後に実行するid jdupracticewriterとid jdupracticeviewerを見直す。最後の2項目がFAILなら、まずP2-PERM-04の原因を確認する。jdu-checkは必ずssm-userで実行する。設定が正しいのにFAILが続く場合は、リセットせず、FAILのチェックIDを教員に伝える。

解答例（操作手順）

手順1: 管理userを確認する

```
# ホームディレクトリへ移動する。
cd ~
# 現在のユーザー名を表示する。
id -un
# 現在のユーザーIDと所属グループを表示する。
id
```

手順2: 初期groupを確認する

```
# 共有グループの登録内容を表示する。
getent group practiceops
# 書き込み担当者の所属グループを確認する。
id jdupracticewriter
# 閲覧担当者の所属グループを確認する。
id jdupracticeviewer
```

初期状態ではjdupracticeviewerがpracticeopsに入り、jdupracticewriterが入っていない。

手順3: Group membershipを修正する

```
# ホームディレクトリへ移動する。
cd ~
# 書き込み担当者を共有グループへ追加する。
sudo usermod -aG practiceops jdupracticewriter
# 閲覧担当者を共有グループから外す。
sudo gpasswd -d jdupracticeviewer practiceops
# 変更後の共有グループを表示する。
getent group practiceops
# 書き込み担当者の所属を確認する。
id jdupracticewriter
# 閲覧担当者の所属を確認する。
id jdupracticeviewer
```

手順4: 共有directoryを設定する

```
# /srvへ移動する。
cd /srv
# 現在地を表示する。
pwd
# 共有ディレクトリ自身の初期権限を表示する。
ls -ld jdu-practice-share
```

変更前の権限・所有者・所有グループを確認する。ls -lは詳細表示、-dはディレクトリの中身ではなくディレクトリ自体を表示する指定である。表示の先頭が権限、その後のリンク数に続く2列が所有者と所有グループである。

```
# 共有ディレクトリの所有者とグループを変更する。
sudo chown root:practiceops jdu-practice-share
# 所有者とグループの変更を確認する。
ls -ld jdu-practice-share
# グループ継承を含む権限2775を設定する。
sudo chmod 2775 jdu-practice-share
# setgidのsと権限を確認する。
ls -ld jdu-practice-share
```

chownの後は所有者・所有グループがroot practiceops、chmodの後は権限がdrwxrwsr-xになることを確認する。グループの実行権限の位置にあるsはsetgidを表し、このディレクトリ内に作るファイルが所有グループを引き継ぐ。

手順5: GUIDE.txtを設定する

対象directoryへ移動する。

```
# 共有ディレクトリへ移動する。
cd /srv/jdu-practice-share
# 現在地を表示する。
pwd
# GUIDE.txtの初期権限を表示する。
ls -l GUIDE.txt
```

変更前のファイルの権限・所有者・所有グループを確認してから設定する。

```
# GUIDE.txtの所有者とグループを変更する。
sudo chown root:practiceops GUIDE.txt
# 所有者とグループを確認する。
ls -l GUIDE.txt
# 所有者とグループに読み書きを許可する。
sudo chmod 664 GUIDE.txt
# 最終的なファイル権限を確認する。
ls -l GUIDE.txt
```

所有者・所有グループがroot practiceops、権限が-rw-rw-r--になることを確認する。所有者と所有グループには読み書き、その他のユーザーには読み取りだけを許可している。

手順6: Writerへ切り替えてfileを作る

```
# ホームディレクトリへ戻る。
cd ~
# 書き込み担当者のシェルに切り替える。
sudo su - jdupracticewriter
# 切り替え後のユーザー名を確認する。
id -un
# 切り替え後の所属グループを確認する。
id
# 共有ディレクトリへ移動する。
cd /srv/jdu-practice-share
# 現在地を確認する。
pwd
# 書き込み担当者としてファイルを作る。
printf '%s\n' 'guided writer file' > writer-created.txt
# 作ったファイルの所有グループを確認する。
ls -l writer-created.txt
# 書き込み担当者のシェルを終了する。
exit
# 管理ユーザーに戻ったことを確認する。
id -un
```

最後にssm-userと表示されることを確認する。

手順7: Viewerのreadとwriteを確認する

```
# ホームディレクトリへ戻る。
cd ~
# 閲覧担当者のシェルに切り替える。
sudo su - jdupracticeviewer
# 切り替え後のユーザー名を確認する。
id -un
# 閲覧担当者の所属グループを確認する。
id
# 共有ディレクトリへ移動する。
cd /srv/jdu-practice-share
# 現在地を確認する。
pwd
# 閲覧担当者としてGUIDE.txtを読む。
cat GUIDE.txt
# 書き込みが拒否されることを確認する。
touch viewer-created.txt
# 閲覧担当者のシェルを終了する。
exit
# 管理ユーザーに戻ったことを確認する。
id -un
```

catは成功する。touchはPermission deniedになる。これは想定した失敗である。

jdupracticeviewerは所有者のrootではなく、手順3でpracticeopsからも外したため、「その他のユーザー」の権限が適用される。GUIDE.txtはその他のユーザーにも読み取り権限（r--）があるので、catで読める。一方、共有ディレクトリのその他のユーザーの権限はr-xで、書き込み権限（w）がない。新しいファイルを作るには、そのファイルを置くディレクトリへの書き込み権限と実行権限が必要なので、touchによる作成は拒否される。

手順8: 採点する

```
# ホームディレクトリへ戻る。
cd ~
# 採点するユーザー名を確認する。
id -un
# P2の完成状態を採点する。
jdu-check P2
```

全6件がPASSになったら、M2へ進む。

P3 Processとpackage

第8章を読んだら手順1〜4でプロセスを確認・停止する。続いて第9章を読み、手順5でパッケージを導入する。

学ぶこと

- systemd serviceとprocessのPIDを対応させる。
- Main PIDをpsで照合し、対象のserviceを停止する。
- aptでpackageをinstallする。

課題

初期状態でjdu-p3-process1.service、jdu-p3-process2.service、jdu-p3-process3.serviceが動いている。figletは未インストールである。次のAとBの両方を完了する。

課題A - process2だけを停止する。 jdu-p3-process2.serviceのMain PIDを調べ、そのPIDのプロセスをpsで確かめる。対象のserviceをsystemctl stopで停止する。process2だけが停止し、process1とprocess3は動いたままにする。

課題B - パッケージを導入する。 figletのパッケージ情報を調べ、aptでインストールする。コマンドを1回実行する。

jdu-check P3でAとBを各1項目、計2項目として判定する。調査時のコマンド履歴やメモは提出しない。

解答例（操作手順）

手順1: 3 serviceを確認する

```
# ホームディレクトリへ移動する。
cd ~
# P3のサービス一覧を表示する。
systemctl list-units --type=service 'jdu-p3-*'
# process1の状態を表示する。
systemctl status jdu-p3-process1.service --no-pager
# process3の状態を表示する。
systemctl status jdu-p3-process3.service --no-pager
```

手順2: Process2のMain PIDを読む

```
# process2の状態とMain PIDを表示する。
systemctl status jdu-p3-process2.service
```

Main PID:の右に出る番号を手元にメモする。ページャーが開いた場合はqで終了する。

手順3: メモしたPIDのprocessを見る

例えば、メモした番号が1234なら次のように入力する。1234を自分の画面の番号に置き換える。

```
# メモしたPIDのプロセス情報を表示する。1234は自分の値に置き換える。
ps -fp 1234
```

PID、USER、CMDを確認する。

手順4: Process2のserviceを停止する

```
# process2のサービスを停止する。
sudo systemctl stop jdu-p3-process2.service
# process1が稼働中か確認する。
systemctl is-active jdu-p3-process1.service
# process2が停止中か確認する。
systemctl is-active jdu-p3-process2.service
# process3が稼働中か確認する。
systemctl is-active jdu-p3-process3.service
```

順にactive、inactive、activeになる。

手順5: figletを調査してinstallする

```
# ホームディレクトリへ移動する。
cd ~
# figletのパッケージ情報を読む。
apt show figlet
# 利用可能なパッケージ一覧を更新する。
sudo apt update
# figletをインストールする。
sudo apt install -y figlet
# figletコマンドの場所を確認する。
command -v figlet
# figletを一度実行する。
figlet JDU
# P3の完成状態を採点する。
jdu-check P3
```

figlet JDUは文字を表示した後、自動で終了してプロンプトへ戻る。jdu-check P3は課題A・Bを各1件として判定する。

全2件がPASSになったら、M3へ進む。

P4 systemd service

学ぶこと

- Unit fileを読む。
- Activeとenabledを別々に設定する。
- User、WorkingDirectory、ExecStartをunitの設定として読む。

課題

jdu-practice-status.serviceのユニットファイルは完成しているが、サービスは停止中で、自動起動も無効である。ユニットファイルを変更せずに次を行う。

1. ユニットファイルのUser、WorkingDirectory、ExecStartを調べる。
2. サービスを起動し、現在の状態がactiveであることを確認する。
3. 起動時の自動実行を有効にし、enabledであることを確認する。activeとenabledは別の状態として調べる。

jdu-check P4は、変更されていないユニットの稼働状態と自動起動を計2項目で判定する。観察結果を別ファイルへ提出する必要はない。

解答例（操作手順）

手順1: Unit fileを読む

```
# ホームディレクトリへ移動する。
cd ~
# サービスのユニット設定を表示する。
systemctl cat jdu-practice-status.service
```

User、WorkingDirectory、ExecStartを確認する。

手順2: 初期状態を確認する

```
# 現在稼働しているか確認する。
systemctl is-active jdu-practice-status.service
# 起動時の自動実行が有効か確認する。
systemctl is-enabled jdu-practice-status.service
```

手順3: Serviceをstartする

```
# サービスを今すぐ起動する。
sudo systemctl start jdu-practice-status.service
# 現在の稼働状態を確認する。
systemctl is-active jdu-practice-status.service
```

手順4: Boot時にstartするよう設定する

```
# 起動時の自動実行を有効にする。
sudo systemctl enable jdu-practice-status.service
# 自動実行の設定を確認する。
systemctl is-enabled jdu-practice-status.service
```

手順5: 採点する

```
# ホームディレクトリへ移動する。
cd ~
# P4の完成状態を採点する。
jdu-check P4
```

全2件がPASSになったら、M4へ進む。

P5 Port、socket、HTTP、journal

学ぶこと

- Service、process、listening socketを対応させる。
- IP addressとportを読む。
- HTTP requestがjournalへ記録されることを確認する。

課題

jdu-practice-web.serviceは停止中である。ユニットファイルを変更せず、次の一連のつながりを完成させる。

1. ユニットファイルから、実行ユーザー、公開するファイルのパス、待受IPアドレスとポート番号を調べる。サービスを起動する。
2. 127.0.0.1:8181で待ち受けているソケットを見つける。そのソケットのPIDがサービスのMain PIDと同じであることを確かめる。全アドレスでの待受には変更しない。
3. http://127.0.0.1:8181/へリクエストを送り、HTTP 200と本文を確認する。サービスの実行ユーザーが/srv/jdu-practice-web/index.txtを読めることも確かめる。
4. http://127.0.0.1:8181/p5-checkへ自分でリクエストを送り、現在のサービスのシステムジャーナルにREQUEST path=/p5-checkが記録される状態にする。
5. ポート18181には待受がないことを調べ、接続できる8181との違いを確認する。

jdu-check P5は、ソケット、HTTPとファイルアクセス、指定リクエストのログ、開いているポートと閉じているポートの比較を計4項目で判定する。

解答例（操作手順）

手順1: Unit fileを読む

```
# ホームディレクトリへ移動する。
cd ~
# Webサービスのユニット設定を表示する。
systemctl cat jdu-practice-web.service
```

127.0.0.1、8181、jdupracticeweb、content fileのpathを確認する。

手順2: Serviceをstartする

```
# Webサービスを今すぐ起動する。
sudo systemctl start jdu-practice-web.service
# 稼働状態を確認する。
systemctl is-active jdu-practice-web.service
```

手順3: Listening socketを確認する

```
# 8181番ポートの待受アドレスとPIDを表示する。
sudo ss -lntp | grep ':8181'
```

127.0.0.1:8181を確認する。同じ出力で0.0.0.0:8181や[::]:8181の待受がないことも確認する。前者はこのホスト内からの接続だけを受け付ける。後者は全アドレスでの待受を表す。

手順4: Socket PIDとMain PIDを比較する

```
# サービスの状態を表示し、Main PIDの数字を読む。
systemctl status jdu-practice-web.service --no-pager
# 8181番ポートで待ち受けるプロセスのpid=を読む。
sudo ss -lntp | grep ':8181'
```

systemctl statusのMain PID:と、ssのusers:欄にあるpid=の数字が同じであることを確認する。同じなら、このサービスの主プロセスが8181番ポートで待ち受けている。

手順5: HTTP responseを確認する

```
# HTTPの応答ヘッダーと本文を表示する。
curl -i http://127.0.0.1:8181/
```

手順6: Service userのread権限を確認する

Content directoryへ移動する。

```
# 公開ファイルのディレクトリへ移動する。
cd /srv/jdu-practice-web
# 現在地を表示する。
pwd
# ファイルまでの各ディレクトリの権限を表示する。
ls -ld /srv /srv/jdu-practice-web
# 公開ファイルの所有者・グループ・権限を表示する。
ls -l index.txt
# サービス用ユーザーの所属グループを表示する。
id jdupracticeweb
# サービス用ユーザーのログインシェルを表示する。
getent passwd jdupracticeweb
# サービス用ユーザーの権限で一度だけファイルを読む。
sudo -u jdupracticeweb -- cat index.txt
```

`ls -ld`は各directory自身の権限を、`ls -l`は`index.txt`の所有者・group・権限を表示する。ファイルを読むには、ファイルのread権限に加え、そこへ至る各directoryを通過するためのexecute権限も必要である。`id`ではサービス用ユーザーのgroupを、`getent passwd`ではログインシェルを確認する。

`sudo -u jdupracticeweb -- cat index.txt`は、`jdupracticeweb`の権限で`cat index.txt`を一回だけ実行する。`--`は`sudo`のオプションの終わりを示す。内容が表示されれば、そのユーザーはファイルを読めている。`jdupracticeweb`のログインシェルは`/usr/sbin/nologin`なので、通常の`su -`で対話ログインするのではなく、この方法で確認する。

手順7: 学生用requestを送る

```
# 指定したパスへHTTPリクエストを送る。
curl -i http://127.0.0.1:8181/p5-check
# サービスの最近のログを表示する。
sudo journalctl -u jdu-practice-web.service --no-pager -n 20
```

`REQUEST path=/p5-check`を確認する。

手順8: Closed portと比較する

```
# 待受のない18181番ポートへ接続を試す。
curl --max-time 2 http://127.0.0.1:18181/
# 18181番ポートに待受がないことを確認する。
sudo ss -lnt | grep ':18181'
```

どちらも成功しない。TCP 18181は待ち受けられていない。

手順9: 採点する

```
# ホームディレクトリへ移動する。
cd ~
# P5の完成状態を採点する。
jdu-check P5
```

全4件がPASSになったら、M5へ進む。

P6 SSH、remote command、scp

学ぶこと

- CloudShellとUbuntuを区別する。
- scpでuploadとdownloadを行う。
- SSH remote commandを実行する。

課題

CloudShellを接続元、Ubuntuを接続先として作業する。SSHの接続設定とjdu-ubuntuという接続名は準備済みである。P6の転送ファイルはまだない。

リモートで作る結果ファイルは、次の3行の形式にする。右辺は実際のコマンド出力を使い、推測して手入力しない。

```
REMOTE_USER=実際のユーザー名
REMOTE_HOST=実際のホスト名
REMOTE_PATH=実際のホームディレクトリ
```

1. CloudShellの~/jdu-lab/p6/practice-source.txtへ、JDU SSH guided transferの1行だけを書き込む。
2. scpでそのファイルをUbuntuの~/jdu-lab/p6/practice-upload.txtへ送る。転送前後のSHA-256が同じことを確認する。
3. CloudShellからSSHでUbuntu上のコマンドを実行し、Ubuntuの~/jdu-lab/p6/practice-remote-result.txtへ、実際のリモートユーザー・ホスト名・ホームディレクトリを上の形式で保存する。
4. Ubuntuへ接続してjdu-check P6を実行する。Ubuntu側の2項目がPASSになることを確認する。
5. 結果ファイルをscpでCloudShellの~/jdu-lab/p6/practice-downloaded-result.txtへ戻す。両側のSHA-256が同じことを確認する。
6. CloudShellでjdu-check P6を実行する。CloudShell側の4項目がPASSになることを確認する。

P6はUbuntu側2/2とCloudShell側4/4の両方が揃って完了する。教員用進捗画面でも両側を区別して表示する。

解答例（操作手順）

手順1: UbuntuからCloudShellへ戻る

Ubuntuのpromptで実行する。

```
# UbuntuのSSH接続を終了し、CloudShellへ戻る。
exit
```

CloudShellで現在地を確認する。

```
# CloudShellのホームディレクトリへ移動する。
cd ~
# CloudShell側の現在地を表示する。
pwd
# CloudShell側のユーザー名を表示する。
id -un
# CloudShell側のホスト名を表示する。
hostname
```

手順2: CloudShell側の作業directoryを作る

```
# CloudShellにP6の作業ディレクトリを作る。
mkdir -p ~/jdu-lab/p6
# CloudShellのP6ディレクトリへ移動する。
cd ~/jdu-lab/p6
# 現在地を確認する。
pwd
```

手順3: Upload元fileを作る

```
# 転送元ファイルへ指定の1行を書く。
printf '%s\n' 'JDU SSH guided transfer' > practice-source.txt
# 転送元ファイルの内容を表示する。
cat practice-source.txt
# 転送前のSHA-256を表示する。
sha256sum practice-source.txt
```

手順4: Ubuntu側のdirectoryを作る

CloudShellで実行する。

```
# SSH経由でUbuntu側のP6ディレクトリを作る。
ssh jdu-ubuntu 'mkdir -p ~/jdu-lab/p6'
```

手順5: Ubuntuへuploadする

CloudShellのP6 directoryにいる状態で実行する。

```
# CloudShellのP6ディレクトリへ移動する。
cd ~/jdu-lab/p6
# scpでCloudShellからUbuntuへファイルを送る。
scp practice-source.txt jdu-ubuntu:~/jdu-lab/p6/practice-upload.txt
# Ubuntu上の転送後ファイルのSHA-256を表示する。
ssh jdu-ubuntu 'sha256sum ~/jdu-lab/p6/practice-upload.txt'
# CloudShell上の転送元ファイルのSHA-256を表示する。
sha256sum practice-source.txt
```

2つのSHA-256が同じであることを確認する。

手順6: SSH remote commandで結果fileを作る

CloudShellで、次の1 commandをそのまま実行する。

```
# Ubuntu上で3つの実測値を結果ファイルへ保存する。
ssh jdu-ubuntu 'cd ~/jdu-lab/p6 && printf "REMOTE_USER=%s\nREMOTE_HOST=%s\nREMOTE_PATH=%s\n" "$(id -un)"
"$(hostname)" "$HOME" > practice-remote-result.txt'
```

内容をremoteで確認する。

```
# SSH経由でUbuntu上の結果ファイルを表示する。
ssh jdu-ubuntu 'cat ~/jdu-lab/p6/practice-remote-result.txt'
```

手順7: Ubuntu側を採点する

Ubuntuへ接続する。

```
# CloudShellからUbuntuへ接続する。
ssh jdu-ubuntu
# Ubuntu側のP6ディレクトリへ移動する。
cd ~/jdu-lab/p6
# Ubuntu側の現在地を表示する。
pwd
# 転送したファイルと結果ファイルを確認する。
ls -l
# Ubuntu側のP6を採点する。
jdu-check P6
# UbuntuのSSH接続を終了し、CloudShellへ戻る。
exit
```

Ubuntu側2件がPASSになる。

手順8: Result fileをdownloadする

CloudShellで実行する。

```
# CloudShellのP6ディレクトリへ移動する。
cd ~/jdu-lab/p6
# scpでUbuntuからCloudShellへ結果ファイルを戻す。
scp jdu-ubuntu:~/jdu-lab/p6/practice-remote-result.txt practice-downloaded-result.txt
# ダウンロードした内容を表示する。
cat practice-downloaded-result.txt
# CloudShell上のダウンロード後ファイルのSHA-256を表示する。
sha256sum practice-downloaded-result.txt
# Ubuntu上の転送元ファイルのSHA-256を表示する。
ssh jdu-ubuntu 'sha256sum ~/jdu-lab/p6/practice-remote-result.txt'
```

2つのSHA-256が同じであることを確認する。

手順9: CloudShell側を採点する

```
# CloudShellのP6ディレクトリへ移動する。
cd ~/jdu-lab/p6
# CloudShell側のP6を採点する。
jdu-check P6
```

CloudShell側4件がPASSになったら、M6へ進む。

学習順序

教科書の章は授業回数と一対一ではない。理解した範囲から演習へ進み、必要なら章へ戻る。

- 第1〜3章と第4章の編集の基本を読む。その後、P0の手順で実機のOSを観察する。
- 第4・5章を読んでP1、M1へ進む。
- 第6・7章を読んでP2、M2へ進む。
- 第8章を読んでP3手順1〜4を行う。第9章を読んでP3手順5を行い、最後にM3へ進む。
- 第10章を読んでP4、M4へ進む。第11章を読んでP5、M5へ進む。
- 第12章を読んでP6、M6へ進む。既習の操作を組み合わせてM7へ進む。

演習全体では次の順序を推奨する。

```
P0 → P1 → M1 → P2 → M2 → P3 → M3 → P4 → M4
  → P5 → M5 → P6 → M6 → M7（統合課題）
```

Pは問題文だけで試してから解答例を見ることも、解答例を見ながら進めることもできる。M1〜M6ではPの手順を見ずに、必要なcommandを自分で選ぶ。M7では、M2、M4、M5で学んだ内容を統合する。